

Phát hiện bất thường trong tiến trình của hệ điều hành Linux

Mai Anh Khoa
Ho Chi Minh City, Vietnam
MSSV: 23520744
Email: 23520744@gm.uit.edu.vn
GitHub: c1khoa

Võ Tạ Hữu Huy
Ho Chi Minh City, Vietnam
MSSV: 23520655
Email: 23520655@gm.uit.edu.vn
GitHub: NickKurriyama

Tóm tắt nội dung—Việc phát hiện tiến trình bất thường [1] trên Linux đóng vai trò quan trọng trong bảo mật hệ thống. Các phương pháp truyền thống như rule-based [2] hoặc thống kê ngưỡng thường gặp hạn chế trong việc phát hiện tiến trình mới và không khai thác các mối quan hệ phức tạp giữa các đặc trưng tiến trình. Trong nghiên cứu này, chúng tôi đề xuất một phương pháp phát hiện bất thường dựa trên phân tích hành vi và cấu trúc tiến trình kết hợp với các mô hình máy học, sử dụng bộ dữ liệu OS Anomaly Detection [3] từ Kaggle. Sau khi tiến hành tiền xử lý và lựa chọn đặc trưng, các mô hình được huấn luyện để phát hiện tiến trình bất thường. Kết quả thử nghiệm cho thấy mô hình tốt nhất đạt độ chính xác 0.90, xếp top 5 trên bảng xếp hạng Kaggle, chứng tỏ tiềm năng ứng dụng cao trong giám sát tiến trình và bảo mật hệ thống Linux.

Từ khóa—bất thường, phát hiện tiến trình, Linux, hệ điều hành, máy học, bảo mật hệ thống, phần mềm độc hại, giám sát tiến trình

I. Giới thiệu

Hệ điều hành Linux [4] hiện nay là lựa chọn phổ biến cho các doanh nghiệp, tổ chức và nhà phát triển phần mềm nhờ tính ổn định, bảo mật, khả năng quản lý hệ thống, lập trình, và hỗ trợ phong phú các phần mềm mã nguồn mở. Linux được triển khai rộng rãi trên các máy chủ, hệ thống đám mây, thiết bị nhúng, cũng như trong các môi trường phát triển và thử nghiệm phần mềm, nhờ khả năng tùy biến cao và cộng đồng hỗ trợ lớn. Khả năng quản lý tiến trình, tài nguyên và quyền truy cập người dùng giúp Linux phù hợp với các môi trường đòi hỏi độ tin cậy cao và giảm thiểu rủi ro bảo mật, khiến nó trở thành nền tảng lý tưởng cho cả doanh nghiệp vừa và nhỏ lẫn các tổ chức lớn trong việc triển khai các ứng dụng quan trọng và hệ thống nhạy cảm.

Cùng với sự phổ biến này, việc giám sát và phát hiện tiến trình bất thường trở nên ngày càng quan trọng. Những tiến trình này có thể xuất hiện dưới nhiều hình thức, bao gồm phần mềm độc hại, tiến trình chiếm dụng CPU, bộ nhớ hoặc I/O quá mức, cũng như các hành vi bất thường khác gây suy giảm hiệu năng, treo hệ thống hoặc tạo lỗ hổng bảo mật. Nếu không được phát hiện kịp thời, chúng có thể dẫn đến mất dữ liệu, gián đoạn dịch vụ hoặc tạo điều kiện cho các cuộc tấn công phức tạp hơn. Việc giám sát liên tục và phát hiện sớm tiến trình bất thường giúp giảm thiểu rủi ro, bảo vệ dữ liệu

nhạy cảm, đảm bảo hệ thống ổn định, tối ưu hiệu năng và nâng cao khả năng phản ứng trước các mối đe dọa bảo mật.

Các phương pháp truyền thống như rule-based hoặc thống kê ngưỡng thường hạn chế trong việc phát hiện các tiến trình bất thường mới, do chỉ dựa vào quy tắc hoặc ngưỡng cố định và không khai thác các mối quan hệ phức tạp giữa các đặc trưng tiến trình. Để khắc phục những hạn chế này, bài nghiên cứu đề xuất một phương pháp kết hợp phân tích hành vi và cấu trúc tiến trình với các mô hình máy học. Phương pháp sử dụng các đặc trưng như hành vi, mối quan hệ cha-con, ID tiến trình, luồng, người dùng và giá trị trả về, được tiền xử lý và lựa chọn đặc trưng trước khi huấn luyện mô hình nhằm phát hiện tiến trình bất thường hiệu quả. Kết quả cho thấy mô hình có thể tích hợp vào hệ thống doanh nghiệp để phát hiện kịp thời các hành vi bất thường, giúp bảo đảm hiệu năng, bảo mật và quản lý tài nguyên hệ thống.

Bài nghiên cứu được cấu trúc như sau: Phần II trình bày và phân tích sơ bộ về tập dữ liệu sử dụng; Phần III mô tả các phương pháp tiền xử lý dữ liệu và kỹ thuật chọn lọc, trích xuất đặc trưng; Phần IV giới thiệu các mô hình máy học được áp dụng, từ việc lựa chọn mô hình phân loại đến tìm kiếm tham số tối ưu; Phần V trình bày đánh giá kết quả dựa trên các thước đo tiêu chuẩn; và Phần VI rút ra kết luận cùng hướng nghiên cứu trong tương lai.

II. Bộ dữ liệu

A. OS Anomaly Detection

Bộ dữ liệu OS Anomaly Detection (Kaggle, 04/2025) được xây dựng để nghiên cứu phát hiện tiến trình bất thường trên Linux. Dữ liệu được thu thập qua *kernel-level monitoring*, mỗi dòng đại diện cho một snapshot hành vi tiến trình hoặc một lần thực thi system call. Bộ dữ liệu gồm `train.csv` (có nhãn), `test.csv` (không có nhãn) và `submission_sample.csv`.

Bộ dữ liệu gồm 763,144 bản ghi với 8 đặc trưng chính, không có giá trị khuyết. Mục tiêu của bài toán là phân loại nhị phân tiến trình thành bình thường hoặc bất thường dựa trên các đặc trưng này:

- **processId**: Mã định danh của tiến trình (PID), dùng để nhận diện từng tiến trình riêng lẻ.
- **threadId**: ID của luồng đang thực thi, có thể trùng PID nếu tiến trình đơn luồng.

- **parentProcessId**: PID của tiến trình cha tạo ra tiến trình hiện tại, phản ánh mối quan hệ cha-con giữa các tiến trình.
- **userId**: ID người dùng thực thi tiến trình, với 0 biểu thị user *root*.
- **mountNamespace**: Mã định danh namespace của tiến trình, thể hiện môi trường cách ly mount.
- **argsNum**: Số lượng tham số được truyền vào lệnh hoặc system call, phản ánh mức độ phức tạp của hành vi tiến trình.
- **returnValue**: Giá trị trả về của system call, cho biết lệnh có thực hiện thành công hay thất bại.
- **target**: Nhãn mục tiêu (0 = bình thường, 1 = bất thường), chỉ có trong *train.csv*, dùng để huấn luyện và đánh giá mô hình.

B. Phân tích dữ liệu

Trước khi áp dụng cho mô hình máy học, chúng tôi tiến hành phân tích dữ liệu sơ bộ để hiểu rõ các đặc trưng và mối quan hệ với nhãn mục tiêu.

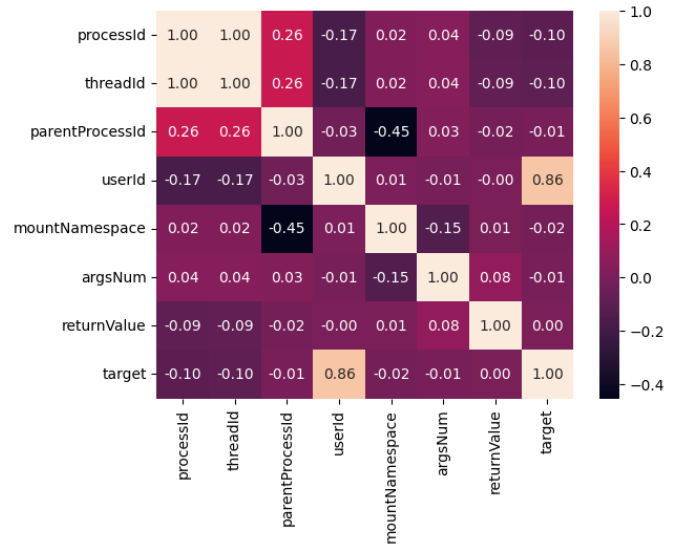
a) *Phân phối nhãn*: Bộ dữ liệu hiện tại bị mất cân bằng nghiêm trọng, với các mẫu bình thường chiếm đa số, trong khi các mẫu bất thường chỉ chiếm tỷ lệ rất nhỏ. Đây là đặc điểm phổ biến trong các bài toán Phát hiện bất thường, bởi vì trong thực tế, hành vi bất thường hiếm khi xảy ra so với hoạt động bình thường của hệ thống.

b) *Ảnh hưởng của userId*: Quan sát cho thấy các tiến trình được thực thi bởi $userId \geq 1000$ hầu hết đều là bất thường, do người dùng bình thường hiếm khi chạy các tiến trình nặng hoặc có hành vi bất thường dẫn đến CPU hoạt động quá mức. Tuy nhiên, các *userId* thấp cũng có thể xuất hiện hành vi bất thường. Điều này cho thấy *userId* là một yếu tố quan trọng trong việc phát hiện bất thường.

c) *Đặc điểm phân bố của argsNum*: Phân tích số lượng *argsNum* cho thấy phần lớn các hàm trong tiến trình có từ 1 đến 4 tham số, thể hiện rằng đây là những hàm quan trọng và phổ biến trong hệ thống, được thiết kế vừa đủ để đảm bảo tính linh hoạt, dễ quản lý và bảo trì. Trong khi đó, các hàm có từ 5 tham số trở lên xuất hiện với tần suất thấp do tính phức tạp, khó mở rộng và khó duy trì, nên ít được gọi trong thực tế. Các hàm không có tham số ($argsNum = 0$) cũng chiếm tỷ lệ nhỏ, bởi chúng thường có mức độ tái sử dụng hạn chế và chỉ phù hợp cho các tác vụ tính toán đơn giản. Nhìn chung, phân bố *argsNum* phản ánh cơ chế thiết kế của hệ thống Linux và cung cấp thông tin quan trọng cho việc lựa chọn, chuẩn hóa đặc trưng trong quá trình tiền xử lý dữ liệu trước khi huấn luyện mô hình máy học.

d) *Ma trận tương quan*: Phân tích tương quan giữa các đặc trưng cho thấy:

- *userId* có ảnh hưởng lớn nhất tới nhãn *target* (hệ số tương quan ~ 0.86), chủ yếu do các $userId \geq 1000$ chiếm một tỷ lệ rất nhỏ nhưng tất cả đều trả về trạng thái bất thường.
- *processId* và *threadId* gần như tuyến tính hoàn toàn (~ 1.00), tức hai biến này trùng khớp hầu hết, nên một trong hai có thể bị loại bỏ để tránh trùng lặp thông tin.



Hình 1. Ma trận tương quan

- *parentProcessId* và *mountNamespace* có tương quan âm vừa phải (~ -0.45), cho thấy các tiến trình cha thường thuộc các namespace khác nhau.

Kết quả phân tích này cung cấp cái nhìn tổng quan về phân bố nhãn, vai trò quan trọng của *userId* và mối quan hệ giữa các đặc trưng, từ đó hỗ trợ quá trình tiền xử lý và lựa chọn đặc trưng trước khi huấn luyện mô hình máy học.

III. Tiền xử lý và Trích xuất đặc trưng

A. Tiền xử lý

a) *Bản ghi trùng lặp*: Chúng tôi quyết định giữ lại các bản ghi trùng lặp vì tần suất xuất hiện của các tiến trình bất thường và bình thường có sự khác biệt rõ rệt, mô hình sẽ học và phân biệt tốt.

b) *Chuẩn hóa*: Các đặc trưng số được chuẩn hóa bằng phương pháp *StandardScaler* để đưa về phân phối chuẩn, giúp các mô hình học máy không bị ảnh hưởng bởi các khoảng giá trị khác nhau và cải thiện hiệu quả huấn luyện.

$$x_{\text{new}} = \frac{x - \mu}{\sigma}$$

trong đó μ là giá trị trung bình và σ là độ lệch chuẩn của đặc trưng x trên tập dữ liệu huấn luyện.

B. Trích xuất đặc trưng

Để nâng cao khả năng phát hiện tiến trình bất thường, chúng tôi tiến hành trích xuất các đặc trưng bổ sung từ dữ liệu gốc. Các đặc trưng mới được thiết kế nhằm phản ánh mối quan hệ, quyền hạn và hành vi của tiến trình, ngoài các đặc trưng cơ bản như mã tiến trình, mã tiến trình cha, mã người dùng, không gian mount, số lượng tham số và giá trị trả về.

a) *Đặc trưng liên quan đến quyền và cấu trúc tiến trình*: Các đặc trưng nhị phân được tạo ra để xác định tiến trình gốc, tiến trình con, cũng như kiểm tra sự trùng khớp giữa luồng và tiến trình. Nhóm này giúp mô hình nhận biết các tiến trình lặp lại hoặc đa luồng.

b) *Đặc trưng tần suất và số lượng duy nhất*: Nhóm này đo lường mức độ hoạt động và sự đa dạng của các thực thể trong hệ thống, bao gồm: số lần xuất hiện của từng người dùng, tiến trình và tiến trình cha; số lượng tiến trình khác nhau do một người dùng hoặc tiến trình cha thực thi; số lượng tiến trình duy nhất trong cùng không gian mount.

c) *Đặc trưng tỷ lệ hành vi*: Các tỷ lệ đặc trưng được tính nhằm phản ánh hành vi đặc thù, bao gồm: tỷ lệ tiến trình gốc, tỷ lệ tiến trình con, và tỷ lệ lỗi trả về. Ngoài ra, số lượng tham số trung bình của các hàm trong tiến trình cũng được tính để đánh giá độ phức tạp của các tiến trình.

d) *Đặc trưng về giá trị trả về*: Các biến nhị phân được tạo từ giá trị trả về để phân biệt các tiến trình thành công, thất bại hoặc trả về giá trị bằng 0.

Nhìn chung, các đặc trưng này cung cấp bối cảnh phong phú về mặt cấu trúc, tần suất, hành vi và kết quả thực thi, giúp mô hình máy học phân biệt tiến trình bình thường và bất thường hiệu quả hơn.

C. Chuẩn bị huấn luyện

Tập dữ liệu huấn luyện được chia thành hai phần: tập huấn luyện (train) và tập xác thực (validation) theo tỉ lệ 8:2. Do dữ liệu chứa nhiều hàng trùng lặp, chúng tôi áp dụng kỹ thuật `GroupShuffleSplit` để đảm bảo tránh hiện tượng rò rỉ dữ liệu và duy trì tỷ lệ nhân cân bằng giữa hai tập.

IV. Mô hình

A. Logistic Regression

Logistic Regression [5] là một thuật toán học máy có giám sát được sử dụng để phân loại dữ liệu. Mục tiêu của mô hình là huấn luyện một bộ phân loại có khả năng đưa ra quyết định nhị phân cho các quan sát đầu vào mới, tức là dự đoán xem một mẫu thuộc về lớp nào trong hai lớp.

Hàm sigmoid được sử dụng để chuyển đổi đầu ra tuyến tính của mô hình thành xác suất nằm trong khoảng từ 0 đến 1. Cụ thể, với một vector đặc trưng $\mathbf{x}$ và vector trọng số $\mathbf{w}$, đầu ra của mô hình được tính bằng:

$$\hat{y} = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}},$$

trong đó $\sigma(\cdot)$ là hàm sigmoid, b là hệ số bias, và $\hat{y}$ biểu diễn xác suất mẫu thuộc về lớp dương.

Với tính chất đơn giản của các siêu tham số trong Logistic Regression, chúng tôi thực hiện huấn luyện và tinh chỉnh mô hình bằng `GridSearchCV`. Quá trình này sử dụng 5-fold cross-validation trên tập huấn luyện, thử nhiều giá trị của các siêu tham số C , $penalty$ và $solver$, với tiêu chí đánh giá là `Average Precision`.

Chúng tôi chọn `Average Precision` làm tiêu chí đánh giá thay vì `accuracy` bởi dữ liệu huấn luyện có sự mất cân bằng nhân đáng kể, với tỉ lệ mẫu thuộc lớp dương rất thấp so với lớp âm. Trong các tình huống như vậy, `accuracy` có thể gây hiểu lầm, vì mô hình chỉ cần dự đoán hầu hết là lớp âm cũng đạt điểm cao, trong khi `Average Precision` đo lường tốt hơn khả năng phát hiện chính xác các mẫu dương, phản ánh

hiệu quả thực sự của mô hình trong việc phát hiện các tiến trình bất thường.

Sau quá trình tìm kiếm, bộ siêu tham số tối ưu được lựa chọn là:

```
{C : 10, penalty : 'l2', solver : 'lbfgs'}.
```

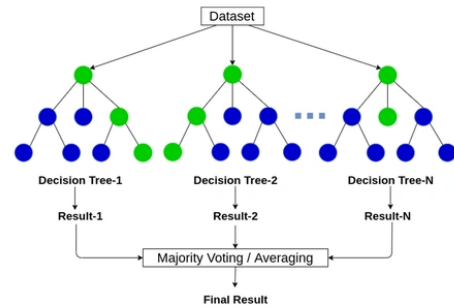
Mô hình với các siêu tham số này được huấn luyện lại trên toàn bộ tập train, và cho kết quả đánh giá trên tập validation, thể hiện khả năng phân loại nhị phân hiệu quả.

B. Random Forest

Random Forest [6] là một thuật toán học máy có giám sát, được sử dụng cho cả phân loại và hồi quy. Mô hình này xây dựng một tập hợp các cây quyết định trên các mẫu dữ liệu khác nhau và các tập con của đặc trưng, nhằm giảm phương sai và hạn chế hiện tượng *overfitting*.

Trong bài toán phân loại, mỗi cây đưa ra một dự đoán nhị phân hoặc đa lớp, và Random Forest tổng hợp kết quả bằng bỏ phiếu đa số để xác định dự đoán cuối cùng. Thuật toán này hoạt động hiệu quả với dữ liệu có nhiều đặc trưng và đặc biệt mạnh trong việc xử lý dữ liệu không tuyến tính hoặc dữ liệu có nhiễu.

Random Forest



Hình 2. Mô phỏng thuật toán Random Forest

Do tính phức tạp của các siêu tham số trong Random Forest, chúng tôi thực hiện tinh chỉnh mô hình bằng `RandomizedSearchCV` kết hợp với 3-fold cross-validation trên tập huấn luyện với số lần chọn là 5, nhằm tiết kiệm thời gian và tài nguyên tính toán. Các siêu tham số được thử gồm: `n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf` và `max_features`.

Sau quá trình tìm kiếm, bộ siêu tham số tối ưu được lựa chọn là:

```
{n_estimators : 200, min_samples_split : 10,
 min_samples_leaf : 4, max_features : 'log2', max_depth : 30}
```

C. XGBoost

XGBoost [7] là một thuật toán boosting dựa trên cây quyết định, sử dụng kỹ thuật *gradient boosting* để kết hợp nhiều

cây yếu thành một mô hình mạnh. Thuật toán được cải tiến với khả năng chuẩn hóa L1/L2 để chống overfitting, tính toán song song để tăng tốc huấn luyện, và xử lý tự động các giá trị khuyết trong dữ liệu.

XGBoost tối ưu một hàm mục tiêu dạng:

$$\mathcal{L}(\phi) = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t)}) + \sum_{k=1}^t \Omega(f_k),$$

trong đó:

- $l(y_i, \hat{y}_i^{(t)})$ là hàm mất mát giữa giá trị thật y_i và dự đoán $\hat{y}_i^{(t)}$ tại vòng lặp t ,
- $\Omega(f_k)$ là hàm phạt độ phức tạp của cây f_k nhằm hạn chế overfitting.

Với bộ siêu tham số phức tạp của XGBoost, chúng tôi sử dụng Optuna để thực hiện tinh chỉnh mô hình. Optuna là một thư viện tối ưu hóa bayesian chuyên dành cho các mô hình có không gian siêu tham số lớn và phức tạp, giúp tìm kiếm hiệu quả các giá trị tối ưu mà không cần thử tất cả tổ hợp. Các siêu tham số được tối ưu bao gồm:

```
{max_depth, learning_rate, min_child_weight, gamma,
subsample, colsample_bytree, reg_alpha, reg_lambda}
```

Sau quá trình tìm kiếm với 50 trials, bộ siêu tham số tối ưu được lựa chọn là:

```
{max_depth : 5, learning_rate : 0.044, min_child_weight : 0.1,
gamma : 2.731, subsample : 0.611, colsample_bytree : 0.969,
reg_alpha : 2.473, reg_lambda : 4.435}
```

D. LightGBM

LightGBM [8] là một mô hình học máy được phát triển dựa trên thuật toán gradient boosting, tương tự XGBoost, nhưng được tối ưu để huấn luyện nhanh hơn và sử dụng bộ nhớ hiệu quả trên các tập dữ liệu lớn.

Thay vì quét toàn bộ dữ liệu, LightGBM sử dụng các kỹ thuật như histogram-based learning và leaf-wise tree growth để tăng tốc quá trình huấn luyện và cải thiện độ chính xác. Phương pháp leaf-wise cho phép cây phát triển sâu hơn ở những nhánh có lỗi lớn nhất, nhờ đó mô hình học được các cấu trúc phức tạp hơn trong dữ liệu. Chúng tôi huấn luyện và tinh chỉnh LightGBM tương tự XGBoost, sử dụng Optuna với 50 trials để tìm bộ siêu tham số tối ưu. Bộ siêu tham số tối ưu được lựa chọn là:

```
{num_leaves : 42, learning_rate : 0.142,
min_child_samples : 57, min_child_weight : 0.073,
subsample : 0.550, colsample_bytree : 0.708, reg_alpha : 4.3,
reg_lambda : 3.111, max_depth : 18, subsample_freq : 4}
```

V. Kết quả nghiên cứu

Sau khi huấn luyện và tinh chỉnh các mô hình học máy, chúng tôi xác định ngưỡng dự đoán tối ưu cho từng mô hình nhằm tối ưu hóa F1. Hiệu năng được đánh giá dựa trên: F1-Score Macro, F1-Score cho lớp 1 (anomaly) tại ngưỡng tốt nhất, và thời gian huấn luyện. Kết quả chi tiết được trình bày trong Bảng I.

Bảng I
Hiệu năng các mô hình học máy

Mô hình	F1 macro	F1 (class 1)	Thời gian (s)
Logistic Regression	0.496	0.011	656.59
Random Forest	0.792	0.584	717.08
XGBoost	0.780	0.561	1605.18
LightGBM	0.844	0.688	1898.18

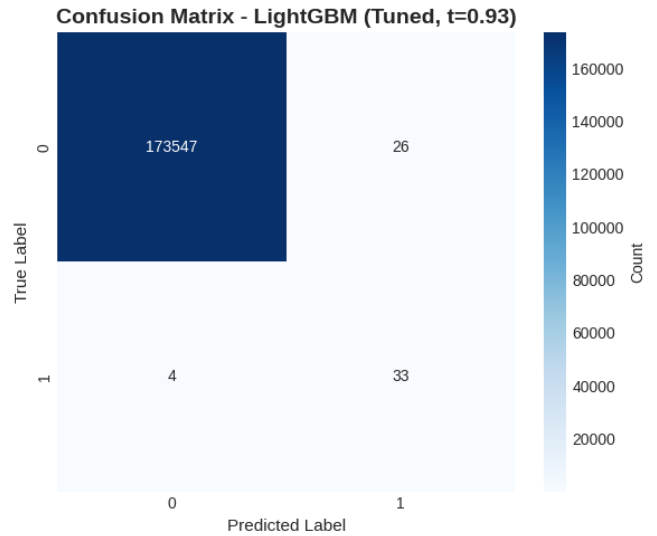
A. Đánh giá từng mô hình

- Logistic Regression: thời gian huấn luyện nhanh nhưng F1 class 1 rất thấp, khả năng phát hiện bất thường bị hạn chế.
- Random Forest: cải thiện F1 class 1 lên 0.584, cân bằng giữa hiệu năng và thời gian huấn luyện.
- XGBoost: F1 class 1 tương tự Random Forest, nhưng thời gian huấn luyện dài hơn.
- LightGBM: F1 Macro và F1 class 1 cao nhất, hiệu quả phân tách lớp tốt nhất, mặc dù thời gian huấn luyện dài nhưng vẫn là lựa chọn tối ưu.

Kết luận:

- Nếu tài nguyên tính toán hạn chế, XGBoost là lựa chọn hợp lý với hiệu năng khá tốt.
- Nếu ưu tiên hiệu năng tối đa (F1 Macro và F1 class 1 cao nhất), LightGBM là lựa chọn tốt nhất cho bộ dữ liệu này, mặc dù thời gian huấn luyện dài hơn.

B. Ma trận nhầm lẫn



Hình 3. Ma trận nhầm lẫn mô hình LightGBM

Hình 3 thể hiện ma trận nhầm lẫn của mô hình LightGBM với ngưỡng tối ưu $t = 0.93$. Mô hình phân loại khá tốt các tiến trình bất thường: trong số 37 tiến trình bất thường thực sự, 33 tiến trình được dự đoán đúng. Tuy nhiên, vẫn có một số tiến trình bình thường bị mô hình nhầm sang lớp bất thường (26 tiến trình), cho thấy LightGBM vẫn có khả năng tạo một số cảnh báo giả, nhưng tỷ lệ này rất thấp so với tổng số tiến trình bình thường (173547).

C. Nộp Kaggle

Chúng tôi đã huấn luyện từng mô hình với các thông số đã tinh chỉnh trên toàn bộ tập huấn luyện và dự đoán trên tập kiểm tra không có nhãn của Kaggle. Kết quả cho thấy LightGBM đạt điểm cao nhất 0.90, nằm trong top 5 trên bảng xếp hạng. Hiệu năng này cho thấy tầm quan trọng của toàn bộ quá trình, bao gồm tiền xử lý dữ liệu, trích xuất đặc trưng và lựa chọn mô hình, trong việc cải thiện khả năng phát hiện tiến trình bất thường.

VI. Hướng phát triển

Qua nghiên cứu, chúng tôi đã huấn luyện và đánh giá thành công các mô hình trên tập dữ liệu OS Anomaly Detection. Kết quả cho thấy phương pháp phát hiện bất thường dựa trên phân tích hành vi và cấu trúc tiến trình, kết hợp với các mô hình máy học, đạt hiệu quả cao.

Tuy nhiên, các kỹ thuật trích xuất đặc trưng và tiền xử lý hiện tại còn đơn giản và thiếu chiều sâu chuyên môn, nên việc mở rộng sang các tập dữ liệu lớn hoặc ứng dụng thực tế có thể gặp khó khăn.

Hiện tại, phương pháp chỉ áp dụng được trên hệ điều hành Linux và chưa triển khai cho Windows hay macOS. Trong tương lai, chúng tôi sẽ nghiên cứu các mô hình Deep Learning và các phương pháp trích xuất đặc trưng nâng cao hơn nhằm cải thiện hiệu quả và khả năng mở rộng.

Tài liệu

- [1] A. S. Yahya, "Supervised machine learning method for anomaly detection," *International Journal of Computer Science and Information Security*, 2019.
- [2] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," *IEEE Symposium on Security and Privacy*, 2010.
- [3] Kaggle, "Data bounty 2: Os anomaly detection," <https://www.kaggle.com/competitions/data-bounty-2-os-anomaly-detection/overview>, 2025.
- [4] Linux Kernel Community, "The linux kernel documentation," <https://www.kernel.org/doc/html/latest/>, 2024.
- [5] D. W. Hosmer and S. Lemeshow, "Introduction to logistic regression," *Applied Logistic Regression*, 2000.
- [6] G. Biau and E. Scornet, "Understanding random forests: From theory to practice," *Statistical Surveys*, 2016.
- [7] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016.
- [8] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.